# eVTOL Patent Taxonomy & Embedding Structure — Progress Report

**Prepared for:** Phase Supervisor **Pipeline stage:** `11_taxonomy_structure_separation.ipynb` (Batches 01 + 05, 1639-patent dataset), model `facebook/dinov2-large` (frozen), layers {18, 22, 24} × pooling {cls, mean_patch}

---

## Section 1: Batches & Architecture Baseline Analysis

> **What this stage checks:** whether the raw review spreadsheets and the downstream processed manifests agree with each other end-to-end (no patents lost or double-counted between review, duplicate-filtering, and architecture extraction), and which DINOv2 layer/pooling configuration the rest of the report will standardize on.

*All figures below are derived directly from the source review files (`Review_postprocess_Batch_01/05.xlsx`, patent-level `T1` section and architecture-level `T2` section, at `…/1639_DS/data/reviewed xlsxs/`) and the downstream processing manifests (`processing_manifest_Batch_01.csv`, `processing_manifest_Batch_05.csv`, at `…/1639_DS/data/processed/Batch_01/` and `…/Batch_05/`) plus the notebook's Stage 0 outputs.*

### 1.1 Individual and Combined Batch High-Level Summary

**Table 1.** Individual and combined batch high-level summary.

| Metric | Batch_01 | Batch_05 | Global / Combined |
|---|---|---|---|
| Total patents reviewed (patent-level, `T1`) | 352 | 211 | **563** |
| Patents disapproved (`isApproved=False`) | 60 | 87[1] | **147** |
| Patents approved (`isApproved=True`) | 292 | 123 | **415** |
| Total unique Patent IDs reviewed *and approved into the architecture stage* (`T2`)[2] | 125 | 95 | **220** |
| Total architecture instances (approved, `T2` "arch" rows) | 301[3] | 230[3] | **531** |
| `is_main == True` images | 150 | 114 | **264** |

[1] Batch_05 has one patent with an unresolved `isApproved` value (blank/incomplete review) — excluded from both the approved and disapproved counts above, so 123 + 87 + 1 = 211 reconciles exactly.

[2] **Why this can be lower than "approved patents":** a patent only reaches the architecture (`T2`) stage after passing patent-level (`T1`) review as approved *and* non-duplicate; some approved-but-duplicate patents (e.g. Batch_01's 183 `isDuplicate=True` patents) are folded into an existing entry rather than creating a new architecture record, which is why 125/95 unique IDs is smaller than 292/123 approved patents.

[3] **Why architecture instances (301/230) exceed unique Patent IDs (125/95):** a single patent can disclose more than one distinct aircraft configuration. Batch_01 has 14 multi-architecture patents (7 with 2 architectures, 3 with 3, 2 with 4, 2 with 5); Batch_05 has 12 (8×2, 2×3, 1×4, 1×5). This is expected and by design — it is *not* a data-quality issue.

**`is_main == True` cross-check:** 264 main images (150 + 114) against 268 total labelled architectures (Section 1.2) reconciles cleanly: 268 − 4 architectures with no approved main image = 264. The two counts match exactly once that known gap is accounted for. *Documented in Stage 0's `qc_issues.csv`.*

#### Duplicate-Type Definitions

The underlying review criteria are identical across both batches — same taxonomy schema (`G1`→`M3`), same image quality checks. When a reviewer flags a patent as a duplicate of one already in the dataset, it is tagged with one of three real categories (exact labels and counts from the `duplicateType` field, `META` section):

**Table 2.** Duplicate-type definitions and counts.

| Type (as labelled in the data) | Batch_01 | Batch_05 | Plain-language meaning |
|---|---|---|---|
| **"1 — Same Aircraft"** | 5 | 6 | Points to an aircraft/design already seen in the dataset, but this patent record is |

| Type (as labelled in the data) | Batch_01 | Batch_05 | Plain-language meaning |
|---|---|---|---|
| | | | otherwise distinct enough to log separately (e.g. different filing). |
| **"2 — Images AND aircraft the same"** | 167 | 27 | True exact duplicate — same aircraft *and* the same drawings; contributes no new visual information. |
| **"3 — Same plane, small changes"** | 10 | – (0) | Same aircraft, but the drawings show minor variations (e.g. a revised figure, small structural tweak) worth keeping separately. |

The three types are defined by which taxonomy fields match between the two records: **Type 1** — `G1` through `M3` are equal. **Type 2** — `T2` through `M3` are equal. **Type 3** — all taxonomy stages differ. Type 2 accounts for the large majority of flagged duplicates in both batches (167 of 182 flagged in Batch_01, 27 of 33 in Batch_05) — most duplicate flags in this dataset are true exact repeats, not near-variants.

## 1.2 Global Batch Pipeline & Data Discrepancy Audit

*Purpose: trace exactly where the 563 reviewed patents narrow down to the 264 images that actually get embedded, and account for every patent/architecture that drops out along the way.*

This subsection tracks the same 563 reviewed patents as they narrow down through the notebook's Stage 0 (`tax.stage0_prepare`) into the final embedded dataset — i.e. where images are gained, dropped, or reshaped, and why.

**Table 3.** Global batch pipeline & data discrepancy audit.

| Pipeline Metric | Value |
|---|---|
| `is_main == True` images (manifest, both batches) | **264** |
| Total architectures labelled | **268** |
| Base patents behind those architectures | 222 |
| Taxonomy attributes (distinct fields) tracked | **256** |
| Images with a fully aligned embedding | **264** |
| Architectures missing an approved main image | 4 |

### Reconciling the Counts

**268 vs. 256:** these describe two different things, not two stages of the same count. **268** is a *row count* — one row per labelled aircraft configuration (architecture). **256** is a *column count* — the number of distinct taxonomy fields recorded per architecture (e.g. `M1_fusShape`, `M2_wingConf`, `M3_boom_bmech`, …). Every architecture row carries all 256 attribute columns (populated or null). *Confirmed via `stage0/coverage_table.csv`, `cov['attribute'].nunique() == 256`.*

**268 vs. 264:** all architecture main images are embedded. The only 4 not embedded have no approved main image on disk, so there is nothing to extract an embedding from for them — `CN106585976A`, `KR102760903B1`, `US2018354616A1`, `US2022348339A1`. 268 − 4 = **264**, matching the `is_main == True` count in Section 1.1 (each of the 264 embedded architectures uses its own main image). *Per `config.yaml`'s `taxonomy.main_arch_only: false` and `src/taxonomy_analysis.py:373-376`; the 4 missing images are listed in `qc_issues.csv` ("architecture without approved main image", n=4); the 264 rows match `aligned_rows.csv` exactly.*

## 1.3 Optimal Layer and Patch Pooling Strategy Selection

*Purpose: compare all 6 candidate DINOv2 layer × pooling configurations and decide which single one the rest of the analysis (Sections 3–4) will standardize on.*

`G1`, `M1`, `M2`, `M3` are section codes in the aircraft-design taxonomy schema — `G1` covers global/topology attributes, `M1` fuselage/gear/boom, `M2` wing/empennage, `M3` propulsion-component attributes. They are used downstream as *labels* to test against clusters (Sections 4.3–4.4). The baseline comparison at this stage of the pipeline is across **DINOv2 layers × pooling method**, evaluated on all 264 embedded images.

**What this table is for — and what it isn't.** Only two of its five columns (effective dim, dims for 90% variance, Hopkins) are actually populated for all 6 candidates; the last two (silhouette, NMI) were only ever run for layer 22 mean_patch, so the table cannot by itself show that configuration winning a head-to-head comparison — on Hopkins alone, layer 24 cls (0.724) and layer 18 mean_patch (0.718) both score higher. The bolded row below is not this table's winner; it is a record of which configuration was carried forward, decided on the criteria described after the table (Section 2.1's cosine-spread analysis), not on this table's numbers.

**Table 4.** Layer × pooling baseline comparison (all 264 embedded images).

| Layer | Pooling | Effective dim (participation ratio) | Dims for 90% variance | Hopkins (clusterability) | Best clustering silhouette[4] | Cluster→taxonomy alignment (best NMI)[4] |
|---|---|---|---|---|---|---|
| 18 | cls | 15.7 | 52 | 0.683 | — | — |
| 18 | mean_patch | 12.2 | 51 | 0.718 | — | — |
| **22** | **mean_patch** | **12.1** | **57** | **0.687** | **0.197 (k-means, k=2)** | **0.260 (M3_emp_chord)** |
| 22 | cls | 20.2 | 61 | 0.706 | — | — |
| 24 | cls | 26.0 | 61 | **0.724** | — | — |
| 24 | mean_patch | 12.1 | 57 | 0.664 | — | — |

[4] Only computed downstream for the pipeline's configured reference matrix (layer 22, mean_patch) — see Sections 4.2/4.3; not run for the other 5 matrices in this baseline pass. *Set in `config.yaml`'s `taxonomy.reference_matrix`.*

**Selection Declaration: Layer 22, mean-patch pooling** is the configuration used for all clustering and taxonomy-alignment work in Sections 3–4. **This choice is not derived from Table 4 above** — on Hopkins score, layer 22 mean_patch (0.687) is actually the second-worst of the six, behind layer 24 cls (0.724) and layer 18 mean_patch (0.718). The justification comes entirely from the cosine-similarity QC histograms in **Section 2.1**, discussed next.

The cosine-similarity QC histograms (Table 5, Section 2.1) are what justify it: layer 22 mean_patch has the most balanced distribution of the six — mean 0.940 with a tight spread (std 0.029). It sits between the two failure modes visible in the other matrices:

- **Too collapsed:** layer 18 (cls mean 0.976, mean_patch 0.942) sits close to 1.0 with little spread — most images look nearly identical in that space, leaving little room for real design differences to show up.
- **Too noisy:** layer 24 cls has the widest spread by far (mean 0.434, std 0.133) — over 10× the spread of layer 22 mean_patch — consistent with the last transformer block's `[CLS]` token being unstable rather than structured.

Layer 22 mean_patch avoids both: enough spread to carry signal, without the instability seen at layer 24 cls. Its Hopkins score (0.687) is close to but not the single highest of the six (layer 24 cls: 0.724) — consistent with this being a stability/separability trade-off, not a case where one matrix dominates on every measure. The taxonomy-alignment result (Section 4.3) then confirms the choice holds up in practice: its best-aligning attribute (`M3_emp_chord`, NMI 0.260) beats every confound tested, including applicant identity (0.207) and drawing perspective (0.201).

**Scope of this selection — best available, not proven optimal, and further investigation is needed.** A caveat worth being explicit about: the cosine-spread argument above cuts both ways. Most matrices here sit close to 1.0 in similarity, which raises a legitimate concern that the embeddings are *too* similar overall — and by that logic a more spread-out matrix (layer 24 mean_patch, mean 0.896, or even layer 24 cls, mean 0.434) could in principle carry more separable design signal, provided the extra spread is structure rather than noise. That question cannot be settled from the histograms alone. What can be said is: layer 22 mean_patch *appeared* strongest on the criteria available at selection time, and the downstream stages were only run on it — so the six matrices were never compared on the criterion that matters most (taxonomy alignment). Section 4.4 provides the first cross-matrix evidence on that front, by measuring attribute separation on all six matrices simultaneously (and in fact finds layer 22 **cls** ahead of layer 22 mean_patch on that criterion — see Section 4.4), and is the natural basis for revisiting this choice in a future iteration. **In short: this selection should be treated as provisional pending that follow-up comparison, not as a settled result.**

*Set as `taxonomy.reference_matrix` in `config.yaml`.*

**Status:** RECONCILED (data audit). **Core finding:** every count reconciles exactly across the pipeline — 563 reviewed → 415 approved → 268 architectures → 264 embedded (the 4-image gap fully explained by missing approved main images) — and layer 22 mean_patch stands as the working reference matrix, chosen on cosine-spread balance rather than proven optimality (see the scope note above and Section 4.4).

DATA AUDIT — RECONCILED

---

# Section 2: Taxonomy Structure Sanity Check

> **What this stage checks:** whether the embeddings themselves are technically sound before drawing any scientific conclusions from them — no corrupted, duplicate, or dead vectors, no NaN/Inf values, across all 6 layer×pooling matrices.

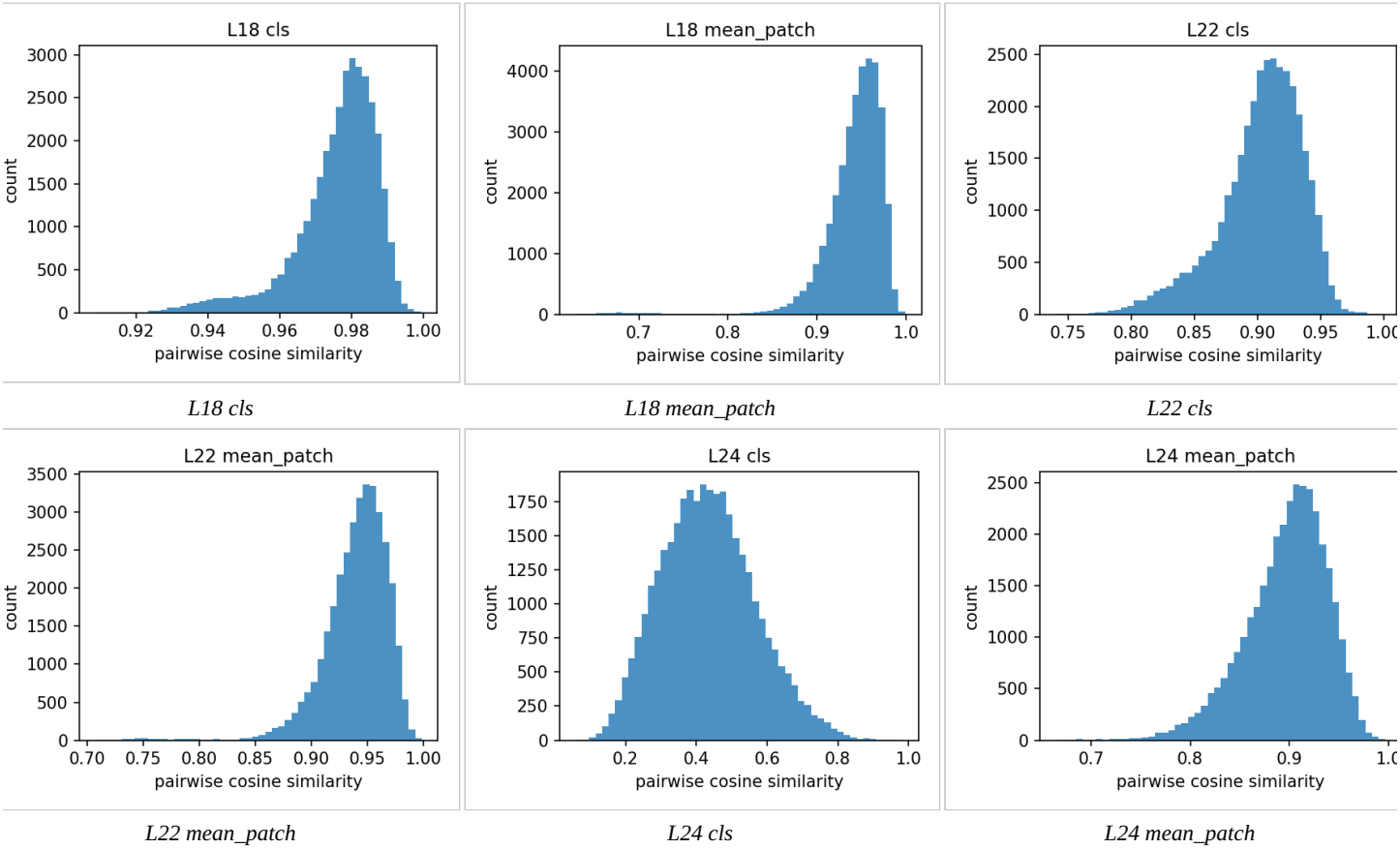
## 2.1 Embedding QC Matrix (Stage 1)

No corrupted/duplicate/dead embeddings were found across any of the 6 layer×pooling matrices (N=264 figures):

**Table 5.** Embedding QC matrix — cosine-similarity statistics and integrity checks, per layer/pooling.

| Layer | Pooling | Cosine mean | Cosine std | Cosine p05 | Cosine p95 | Exact duplicates | Near-dead dims | NaN/Inf |
|---|---|---|---|---|---|---|---|---|
| 18 | cls | 0.9758 | 0.0118 | 0.9509 | 0.9896 | 0 | 0 | 0 |
| 18 | mean_patch | 0.9421 | 0.0348 | 0.8908 | 0.9783 | 0 | 0 | 0 |
| 22 | cls | 0.9029 | 0.0333 | 0.8369 | 0.9478 | 0 | 0 | 0 |
| 22 | mean_patch | 0.9398 | 0.0294 | 0.8905 | 0.9761 | 0 | 0 | 0 |
| 24 | cls | 0.4337 | 0.1332 | 0.2263 | 0.6666 | 0 | 0 | 0 |
| 24 | mean_patch | 0.8964 | 0.0417 | 0.8201 | 0.9541 | 0 | 0 | 0 |

Layer 24 cls is the outlier (mean cosine similarity drops to 0.43) — this is expected for the final block's `[CLS]` token, which specializes heavily late in ViT backbones.




*L18 cls* *L18 mean_patch* *L22 cls*

*L22 mean_patch* *L24 cls* *L24 mean_patch*

***Figure 1.*** *Cosine similarity histograms per matrix (regenerated as individual panels for legibility)*

**How to read these histograms:** each panel is a distribution of pairwise cosine similarity, computed over 500 random pairs of the 264 figures, for one layer/pooling matrix. The x-axis is similarity (1.0 = identical direction in embedding space, 0 = unrelated); the y-axis is how many of the 500 sampled pairs fall at that similarity level. This is purely a QC check, not a design-signal check: it confirms every matrix produces a normal-looking spread with no NaN/Inf, no exact-duplicate spike at 1.0, and no dead dimensions — nothing about *which aircraft look similar* is visible here. A histogram pushed close to 1.0 (e.g. layer 18 cls, mean 0.976) means most images sit close together in that space — little room left for real design differences to separate them. A wider, lower-mean spread (e.g. layer 24 cls, mean 0.43) means images are more spread apart — more room for genuine structure, but also where the embedding is most sensitive to noise. Whether that spread actually reflects real design signal (rather than noise) is what Sections 3–4 test quantitatively, not this QC step.

### 2.2 Sanity Check Validation

**Status:** PASSED (sanity check). **Core finding:** all 6 embedding matrices are technically sound — zero exact duplicates, zero dead/near-dead dimensions, zero NaN/Inf — and the cosine distributions behave as expected per layer depth (near-1.0 early layers, wide spread at layer 24 cls). One observation carried forward rather than resolved here: most matrices concentrate near similarity 1.0, so how much *usable* separation each one actually holds is deferred to the quantitative tests in Sections 3–4.

SANITY CHECK — PASSED

---

# Section 3: Global Embedding Structure vs. Random Noise

> **What this stage checks:** whether the frozen DINOv2 embeddings encode genuine geometric structure, or are statistically indistinguishable from random noise — tested via PCA concentration and pairwise-distance comparison against a random baseline, across all 6 layer/pooling matrices.

### 3.1 PCA Structure Across Layers 18–24, cls vs. mean_patch

**Table 6.** PCA structure across layers 18–24, cls vs. mean_patch.

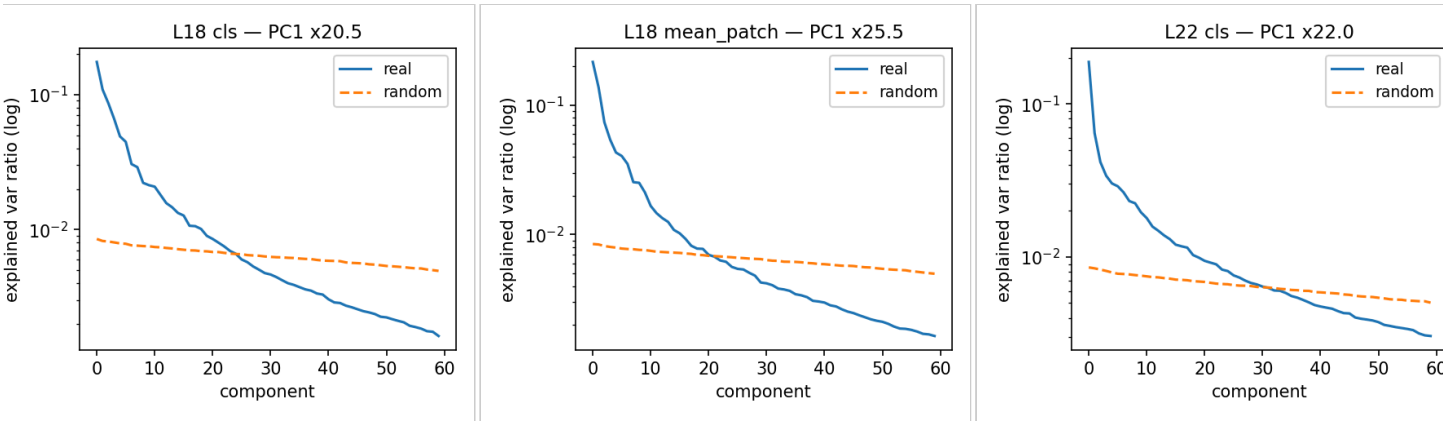
| Layer | Pooling | PC1 variance ratio vs. random | Effective dim | Dims for 90% var |
|---|---|---|---|---|
| 18 | cls | 20.43× | 15.7 | 52 |
| 18 | mean_patch | 25.42× | 12.2 | 51 |
| 22 | cls | 22.01× | 20.2 | 61 |
| 22 | mean_patch | 26.57× | 12.1 | 57 |
| 24 | cls | 18.27× | 26.0 | 61 |
| 24 | mean_patch | **28.49×** | 12.1 | 57 |

`mean_patch` pooling consistently shows a stronger single dominant direction (higher PC1 ratio) than `cls`, while `cls` pooling spreads variance across more effective dimensions (higher participation ratio) — i.e. mean-patch embeddings are more concentrated, cls embeddings are more diffuse.

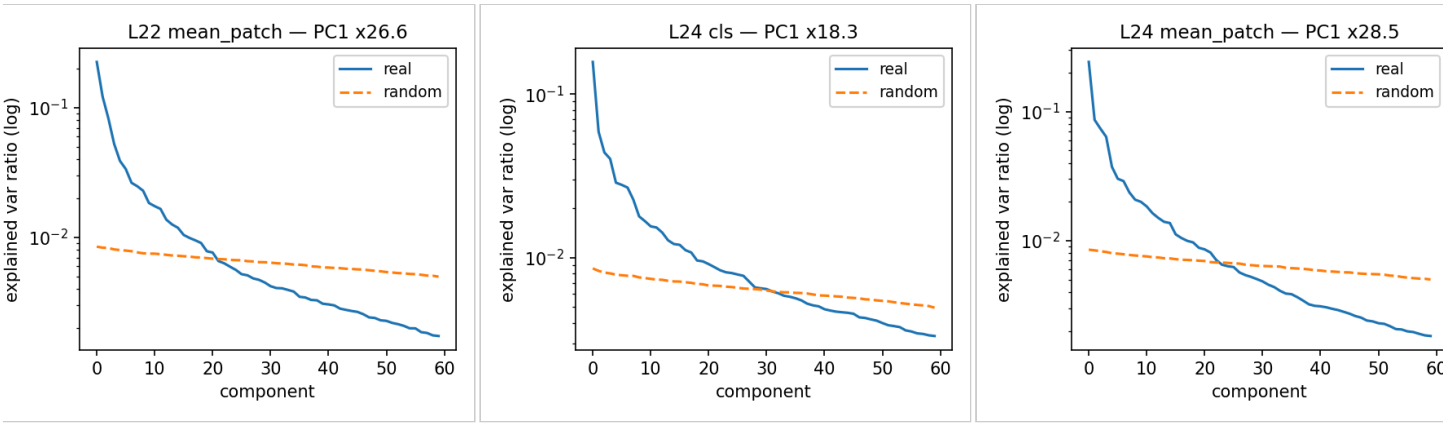
### 3.2 Pairwise Cosine Distance: Real Embeddings vs. Random Baseline

The pipeline's random baseline is a Gaussian-noise matrix of identical shape (N×1024) to each real embedding matrix; PC1 variance ratio (table above) is the primary real-vs-random test, all values comfortably clear the pipeline's >3× pass threshold.

**Figure 2. PCA spectrum, real vs. random (per layer/pooling):**

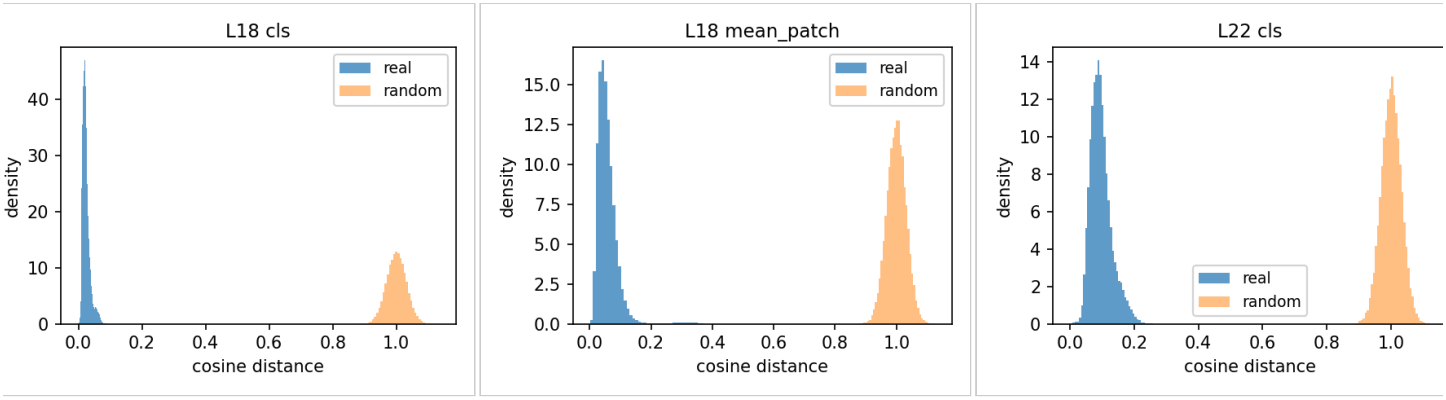



*L18 cls* *L18 mean_patch* *L22 cls*

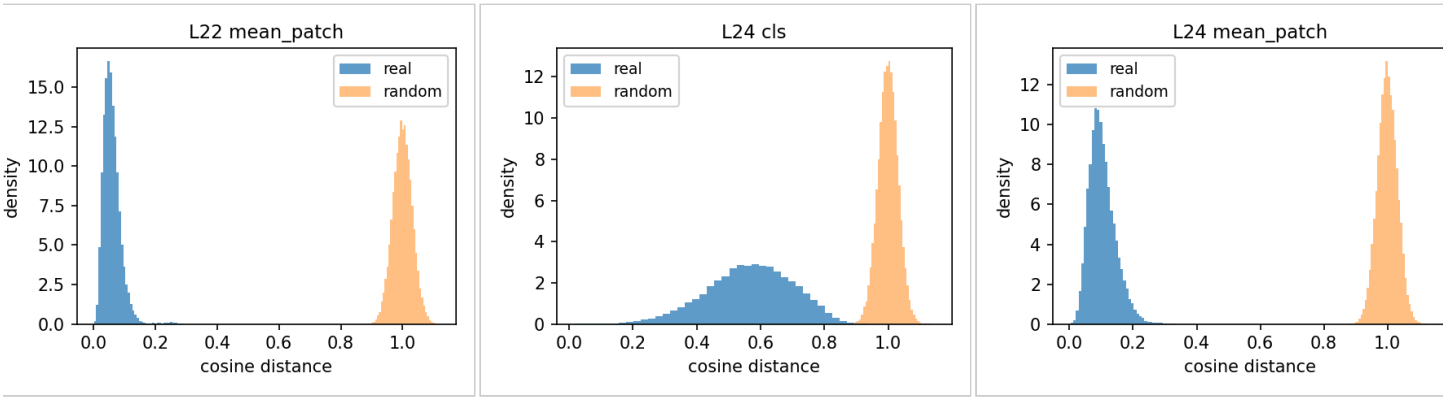



*L22 mean_patch* *L24 cls* *L24 mean_patch*

**Figure 3. Pairwise cosine-distance distribution, real vs. random (per layer/pooling):**




*L18 cls* *L18 mean_patch* *L22 cls*




*L22 mean_patch* *L24 cls* *L24 mean_patch*

## 3.3 Structural Integrity Conclusion

All 6 matrices pass real-vs-random on PC1 concentration (18–28× the random baseline) and show non-trivial local structure (Hopkins ≈ 0.66–0.72, i.e. clearly non-uniform but a smooth continuum rather than sharply separated islands). **Embeddings encode genuine geometric structure, not noise** — but the Hopkins score alone only says the structure is *shaped* like a continuum, not what that continuum is *made of*. That content question is only answered downstream: Section 4.1's branch-level spot-check finds at least one branch (branch 13) grouped by rendering/drawing style rather than aircraft design, and Section 4.3 quantifies drawing perspective as a genuine confound (ARI 0.288, comparable to the strongest real design signal found). Taken together, those two results are what justify reading part of this continuum as drawing-style variation rather than purely design variation — this section's own PCA/Hopkins numbers do not by themselves show that.

**Status:** PASSED (real-vs-random test). **Core finding:** all 6 matrices clear the pipeline's ≥3× threshold by a wide margin (PC1 concentration 18–28× the random baseline), proving the embeddings encode genuine geometric structure rather than statistical noise. The shape of that structure matters for what follows: Hopkins scores of 0.66–0.72 describe a smooth continuum, not sharply separated islands — so Section 4 should be expected to find coarse, gradient-like groupings rather than crisp clusters,

which is exactly what it finds. (What that continuum is made of — design signal vs. drawing style — is a separate question this section cannot answer on its own; see the drawing-style evidence surfaced in Sections 4.1 and 4.3.)

**REAL-VS-RANDOM TEST — PASSED**

---

# Section 4: Unsupervised Image-Level Clustering

> **What this stage checks:** whether unsupervised clustering of the embeddings recovers meaningful aircraft-design groupings, or just superficial drafting/applicant style — by clustering the reference matrix and testing which taxonomy attributes (vs. which confounds) best align with the resulting clusters.

*Using layer 22, mean_patch pooling — the pipeline's configured reference matrix (Section 1.3).*

## 4.1 Hierarchical Dendrogram

*Purpose: visualize how the 264 embedded architectures group hierarchically, as a first qualitative look before any cluster count is chosen.*

Ward-linkage agglomerative clustering (`scipy.cluster.hierarchy`, `method="ward"`) over the 264-figure, layer-22-mean_patch matrix (reduced via PCA to 57 dims per Section 3):

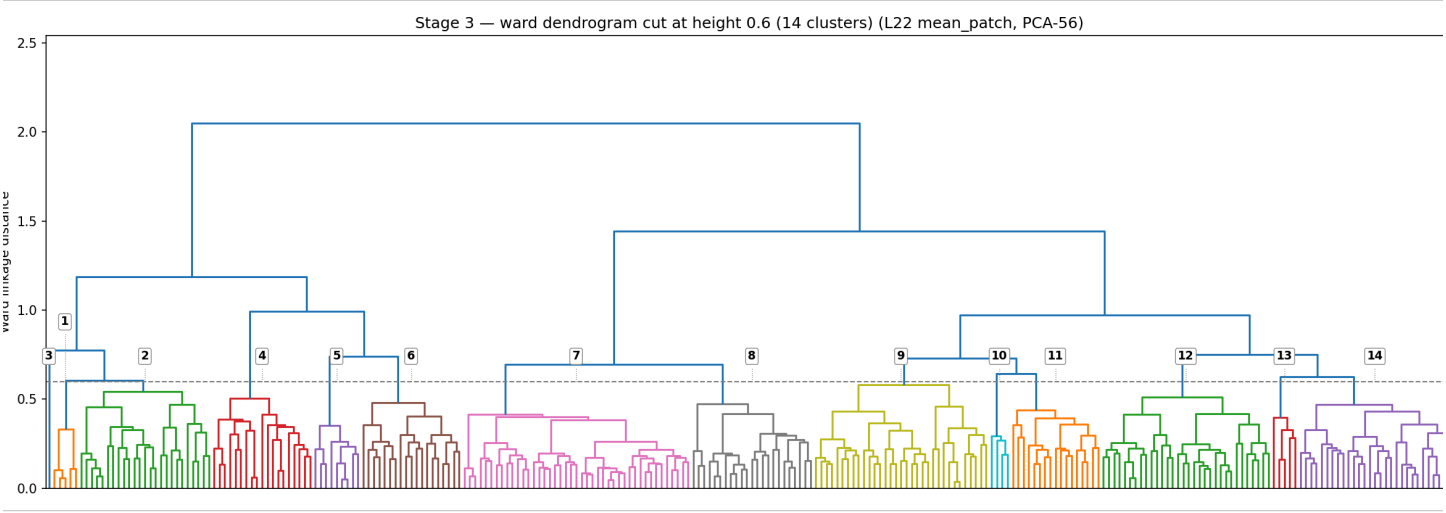



***Figure 4.*** *Dendrogram, labeled 1–14 at the cut used for the branch folders below*

**Branch-level image folders:** the tree was cut at Ward linkage distance 0.60, which is where it splits into exactly **14 branches** (a strict cut at 0.47 gives 21 branches instead, since the natural split points on this tree don't line up with round numbers — 0.60 was chosen to hit 14 exactly). Each labeled branch above corresponds to a folder of that branch's actual main images, for visual auditing:

*`docs/dendrogram_clusters/cluster_01/` … `cluster_14/`, one subfolder per branch, containing that branch's main-image files (branch sizes range from 1 to 43 images; exact counts in `cluster_sizes.json` in the same folder).*

**Visual spot-check of the branches.** Opening representative images from several branches confirms genuine visual consistency, though the common thread differs by branch:

- **Branches 1 and 2** are each internally consistent in drawing perspective, not aircraft design — the common thread within each branch is the viewpoint the figure was drawn from (e.g. same-perspective renders grouped together), not a shared design configuration. Separately, each branch also happens to contain images with similar internal structure/layout to each other, but that consistency tracks perspective, not design.
- **Branch 7** is consistent on a "cruise" configuration: a similar fuselage and main wing shape across its images, with mostly just 2 tilting propulsors rather than many boom-mounted rotors.
- **Branch 14** is the same cruise-style configuration as branch 7, but with more propulsors.
- **Branch 13** is consistent on drawing style, not aircraft configuration: its images are rendered in grayscale/shaded CAD style (solid shaded surfaces) rather than the black-and-white line art used almost everywhere else in the dataset — the branch is really grouping by rendering style, not by design.

That last point echoes the drawing-perspective confound already flagged quantitatively in Section 4.3 (ARI 0.288) — branch 13 is a visual instance of the same effect: an aircraft's drafting/rendering style can pull it into a cluster independent of its actual design.

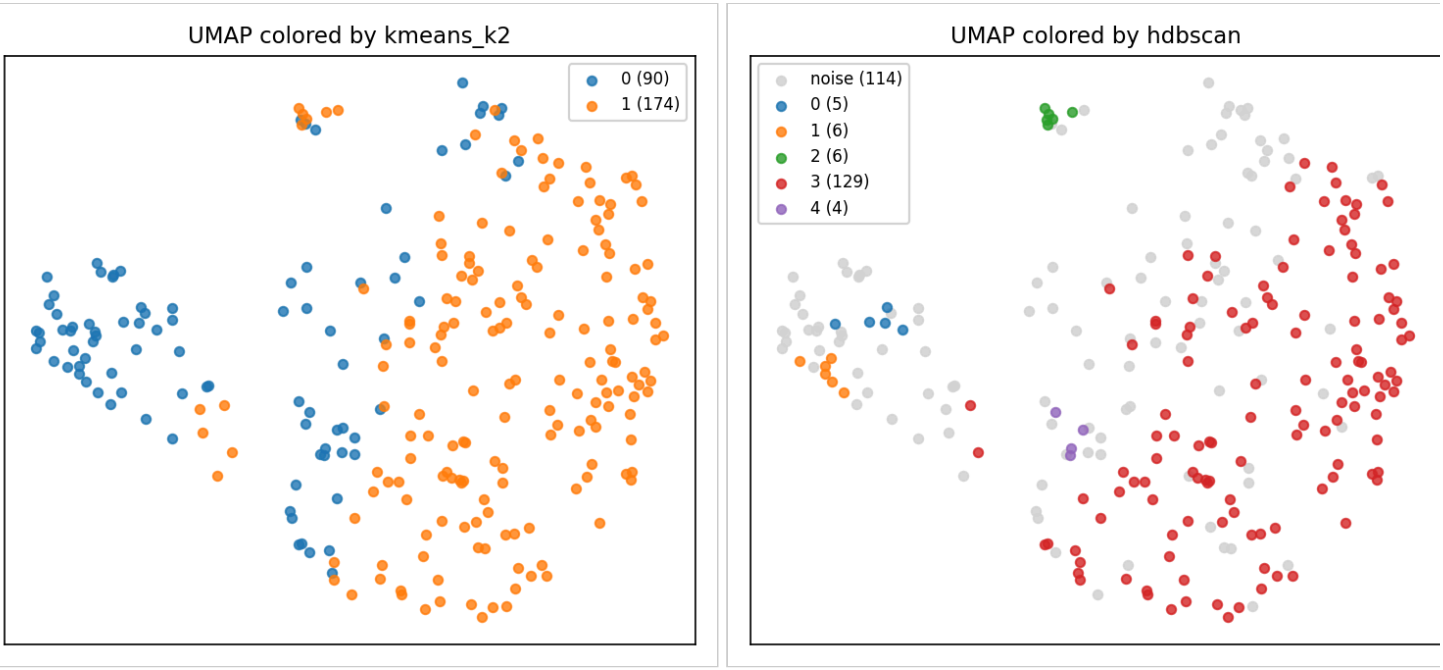
## 4.2 Dimensionality Reduction Comparison

*Purpose: find and validate the actual cluster count/algorithm (k-means, agglomerative, HDBSCAN) that Sections 4.3–4.4 will test against the taxonomy.*

**Conclusions up front:** three things come out of this stage. (1) The embedding space supports a **coarse, stable 2-way split** — every algorithm agrees that k=2 is the best-supported partition (agglomerative silhouette 0.254, k-means 0.197), and that partition is reproducible under resampling (bootstrap ARI 0.724). (2) There is **finer structure below the 2-way split, but it is fragile** — HDBSCAN finds 5 denser sub-groups, at the cost of labeling 43% of images as noise; silhouettes for k = 3–10 all drop below 0.14, so no clean mid-scale cluster count exists in this space yet. (3) Visually, the UMAP projections show the taxonomy attributes are **spread as gradients rather than islands** — colors mix and shade into each other rather than occupying separate regions, matching the "smooth continuum" verdict from Section 3's Hopkins scores.

The projections below are UMAP (2-D layouts of the 264 embeddings, `n_neighbors=15`, `min_dist=0.1`, cosine metric). Two colorings of the *same* layout are shown: (a) by unsupervised cluster assignment — this shows what the clustering algorithms actually found; and (b) by taxonomy attribute — this shows whether human-labelled design categories occupy coherent regions of the space. Reading them side by side is the point: if a taxonomy attribute's colors separate along the same boundary the clusters found, that attribute is a candidate explanation for the cluster split.

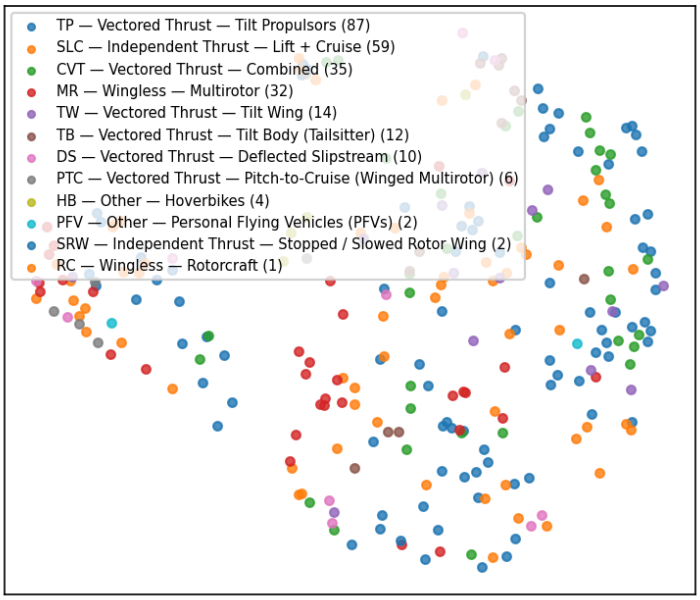
**Figure 5. UMAP colored by unsupervised cluster** (k-means k=2, and HDBSCAN):



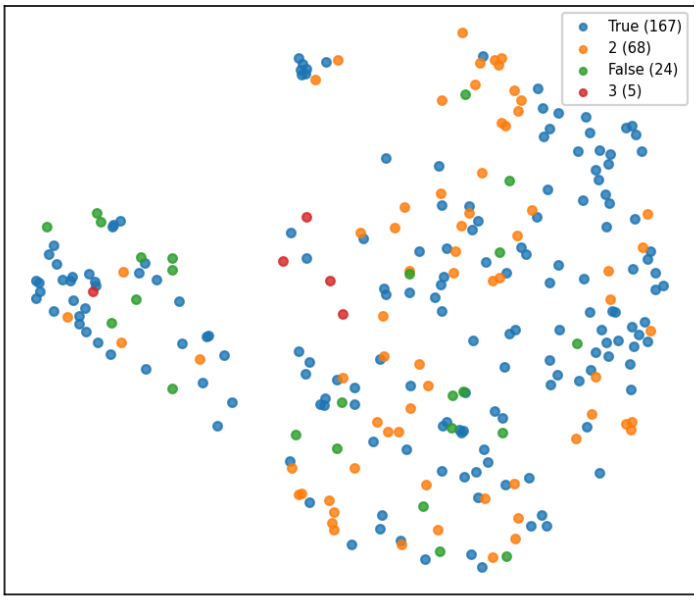

*UMAP — kmeans k=2* *UMAP — HDBSCAN*

The k-means view shows the 2-way split as a left/right frontier (90 vs. 174 images). The HDBSCAN view shows where the *dense cores* are: its largest cluster (129 images) occupies the same right-hand region as one k-means cluster, while the gray "noise" points (114) fill the diffuse space between cores — visual confirmation that the fine structure is real but sparse.

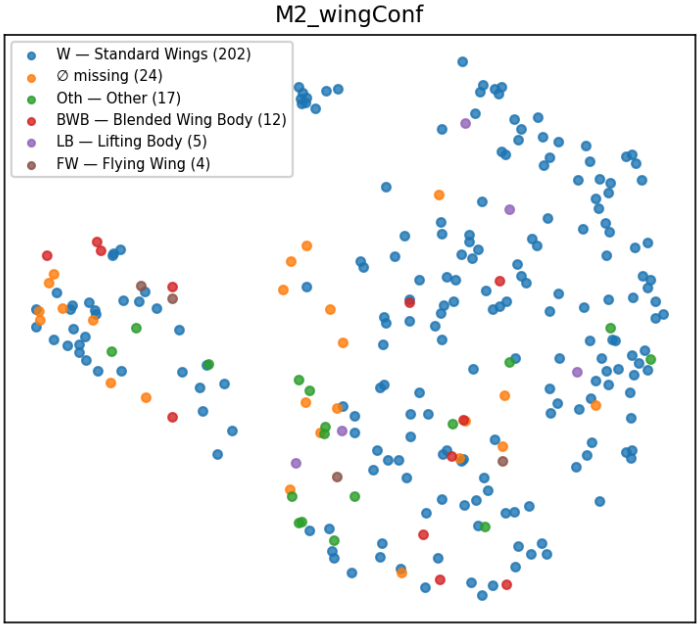
**Figure 6. UMAP colored by taxonomy attribute** (`G1_topType`, `M2_wCount`, `M2_wingConf`, `M1_boomsPresent`, `M1_fusShape`, `M1_gearArch`):



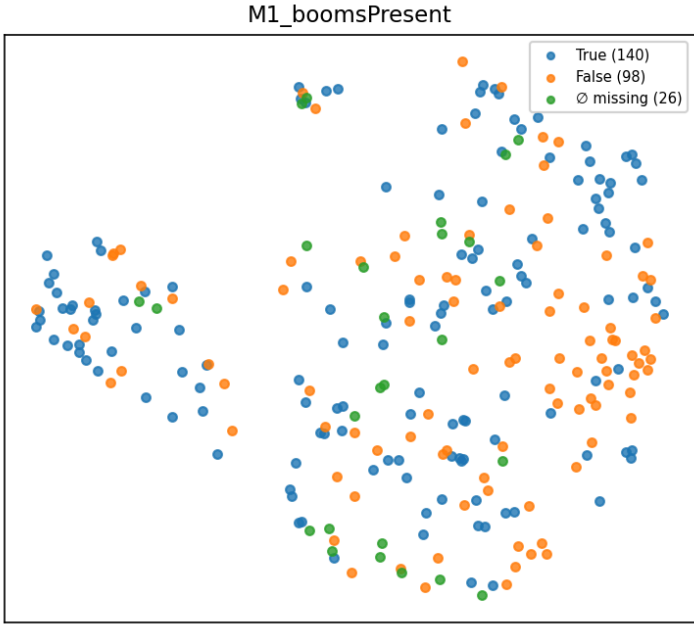

*G1_topType*



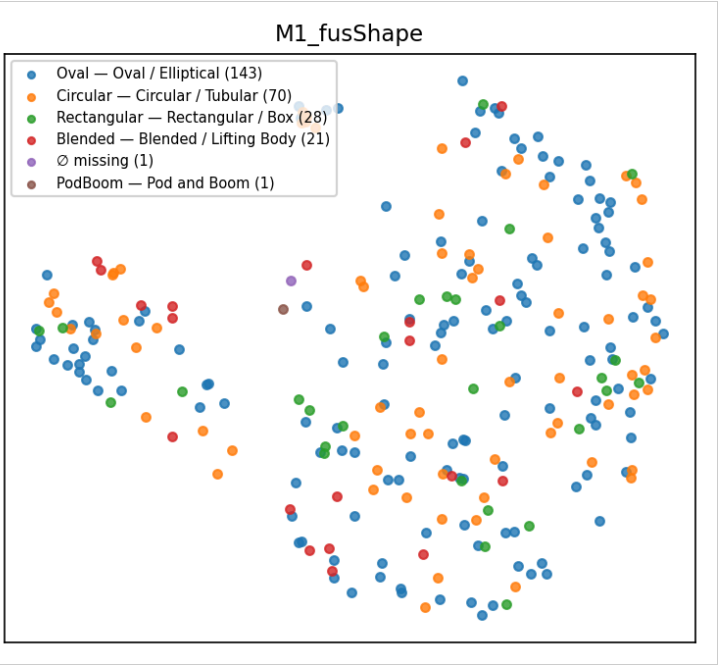

*M2_wCount*



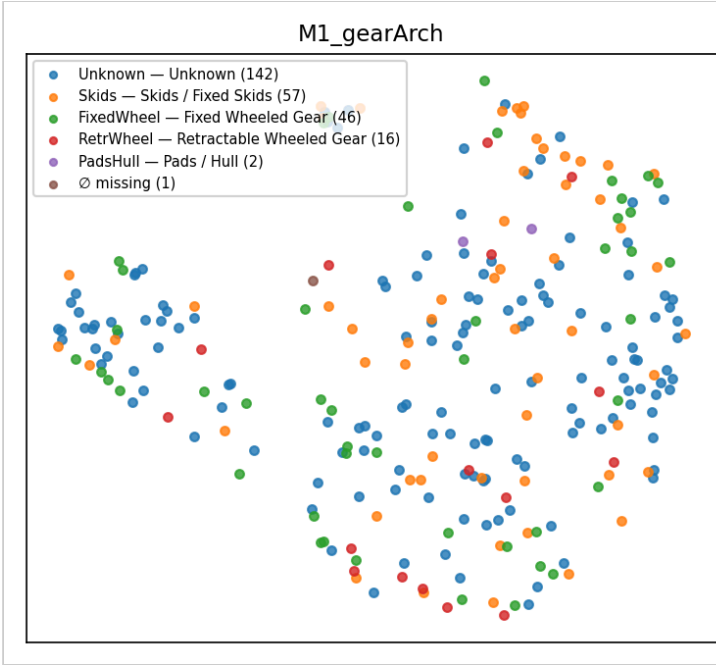

*M2_wingConf*




*M1_boomsPresent*




*M1_fusShape*




*M1_gearArch*

No single attribute cleanly reproduces the cluster boundary on its own — the coloring is mixed everywhere, which is why the quantitative alignment test in 4.3 (rather than visual inspection) is needed to rank which attributes actually track the split.

**Clustering sweep detail:** across k-means / agglomerative / HDBSCAN with k = 2..10, **agglomerative k=2 (silhouette 0.254)** is the best overall; k-means k=2 is close behind (0.197); HDBSCAN settles on 5 clusters with a high noise fraction (0.432, silhouette 0.149) — density-based clustering finds more structure once every architecture is included, but at the cost of calling nearly half the dataset "noise." Bootstrap cluster stability (ARI) = 0.724 for the k-means k=2 partition (up from 0.70 in the main-arch-only run).

## 4.3 Cluster ↔ Taxonomy Alignment (architectures)

*Purpose: the key confound test — check whether the clusters found in 4.2 track real design attributes more strongly than they track nuisance factors like applicant identity or drawing perspective.*

**The two metrics used here, briefly. NMI (Normalized Mutual Information, 0–1)** measures how much knowing an attribute's category (e.g. "this aircraft has a pusher propulsor") reduces uncertainty about which cluster an image landed in — 0 means the attribute tells you nothing about cluster membership, 1 means it determines it completely. Its weakness: it is *not* corrected for chance, so an attribute split into many small categories can score well above 0 by accident. **ARI (Adjusted Rand Index)** measures the same kind of agreement but *is* chance-corrected — 0 is exactly what random labeling would produce, and it can go negative for worse-than-random agreement. Reading the two together is what makes the table trustworthy: high NMI + comparable ARI = real signal; high NMI + near-zero ARI = statistical artifact.

**The "categories / N" column** reads as *(number of possible label values for that attribute) / (number of images that actually carry a non-null value for it)* — e.g. `M3_emp_chord` is `2 / 39`: only 2 possible values ("Front"/"Back"), and only 39 of the 264 embedded images even have this attribute populated, because many taxonomy fields only apply to a subset of aircraft (here, only aircraft with an empennage-mounted propulsor at all). This ratio is precisely what drives NMI's chance-inflation problem: the more label values squeezed into fewer labelled images, the higher NMI drifts on its own.

Best-aligning taxonomy attributes to the 2-cluster (k-means) partition, out of 79 attributes tested:

**Table 7.** Best-aligning taxonomy attributes vs. confounds, 2-cluster (k-means) partition.

| Attribute | NMI | ARI (chance-corrected) | Categories / N labelled | What it is |
|---|---|---|---|---|
| **M3_emp_chord** | **0.260** | 0.218 | 2 / 39 | Whether the empennage-mounted propulsor faces forward or back — puller ("Front") vs. pusher ("Back") configuration. |
| M3_emp_zone | 0.235 | 0.123 | 5 / 41 | Where on the empennage that propulsor sits — tip-mounted, stacked vertically, or stacked horizontally. |
| M3_boom_t2_count | 0.145 | −0.013 | 9 / 94 | Number of secondary (type-2) propulsion units mounted on the boom, |

| Attribute | NMI | ARI (chance-corrected) | Categories / N labelled | What it is |
|---|---|---|---|---|
| | | | | per boom. |
| M2_wing1_posV | 0.139 | 0.210 | 4 / 211 | Vertical mounting position of the main wing on the fuselage — high/shoulder, mid, or low. |
| M1_boom1_circSym | 0.135 | 0.109 | 2 / 111 | Whether the primary boom is circumferentially symmetric (round cross-section) or not. |
| *assignee (confound)* | 0.207 | **0.036** | **109 / 264** | Patent applicant/company — a drafting-style confound, not a design attribute. |
| *drawing perspective (confound)* | 0.201 | 0.288 | 6 / 264 | The figure's viewpoint (front, side, top, isometric, …) — reflects how the patent was drawn, not the aircraft's design. |
| *assignee_country (confound)* | 0.169 | 0.182 | 19 / 261 | Applicant's country of origin — another drafting/institutional confound. |

Cross-referencing the NMI and ARI columns exposes which signals are real and which are artifacts:

- **Assignee (statistical mirage):** having 109 distinct companies across only 264 patents (many appearing just once or twice) inflates its NMI to 0.207. However, its ARI is a mere 0.036, proving the real agreement with clusters is practically zero.
- **M3_boom_t2_count (false positive):** shows a positive-looking NMI (0.145) but a negative ARI (−0.013), meaning it actually aligns worse than random chance.
- **M3_emp_chord (real signal):** with only 2 categories over 39 patents, its ARI (0.218) closely tracks its NMI (0.260). There is no cardinality inflation here; the signal is legitimate.
- **Drawing perspective (genuine confound):** with only 6 categories, its ARI (0.288) is actually higher than its NMI (0.201). This is a genuinely strong relationship, not a mathematical artifact — and, being higher even than `M3_emp_chord`'s ARI, it's the one most worth explicitly controlling for before treating cluster membership as a clean design signal (see next steps).

## 4.4 Intra- vs. Inter-Class Distances (Notebook Stage 5 — layer comparison)

*Purpose: a finer-grained, per-attribute test (independent of any specific cluster count) of whether same-class patents sit closer together in embedding space than different-class patents, and which layer/pooling separates classes best.*

`tax.stage5_distances` tests, per taxonomy attribute and per layer×pooling matrix, whether images sharing the same category value sit closer together in embedding space than images from different categories. **The separation ratio is the mean between-class cosine distance divided by the mean within-class cosine distance**: a ratio of 1.0 means same-class images are no closer to each other than to anything else (the attribute is invisible to the embedding), while e.g. 1.54 means different-class pairs are on average 54% farther apart than same-class pairs. Cohen's d expresses the same gap as a standardized effect size, and the permutation p-value checks it isn't a shuffling accident. Across all 264 figures × 79 attributes tested (492 attribute×layer rows total): **174 rows separate significantly at p < 0.05** (excluding the `cluster` label itself).

One row in this test is not a taxonomy attribute: **`cluster` is the unsupervised k-means assignment from Section 4.2**, fed back through the same separation test. It serves as a reference ceiling — since the clusters were *fit* to minimize exactly this kind of within-group distance on the reference matrix, they should separate more strongly than any human label there, and they do (see below). If any taxonomy attribute ever approached the `cluster` row's separation, that attribute would essentially *be* the clustering.

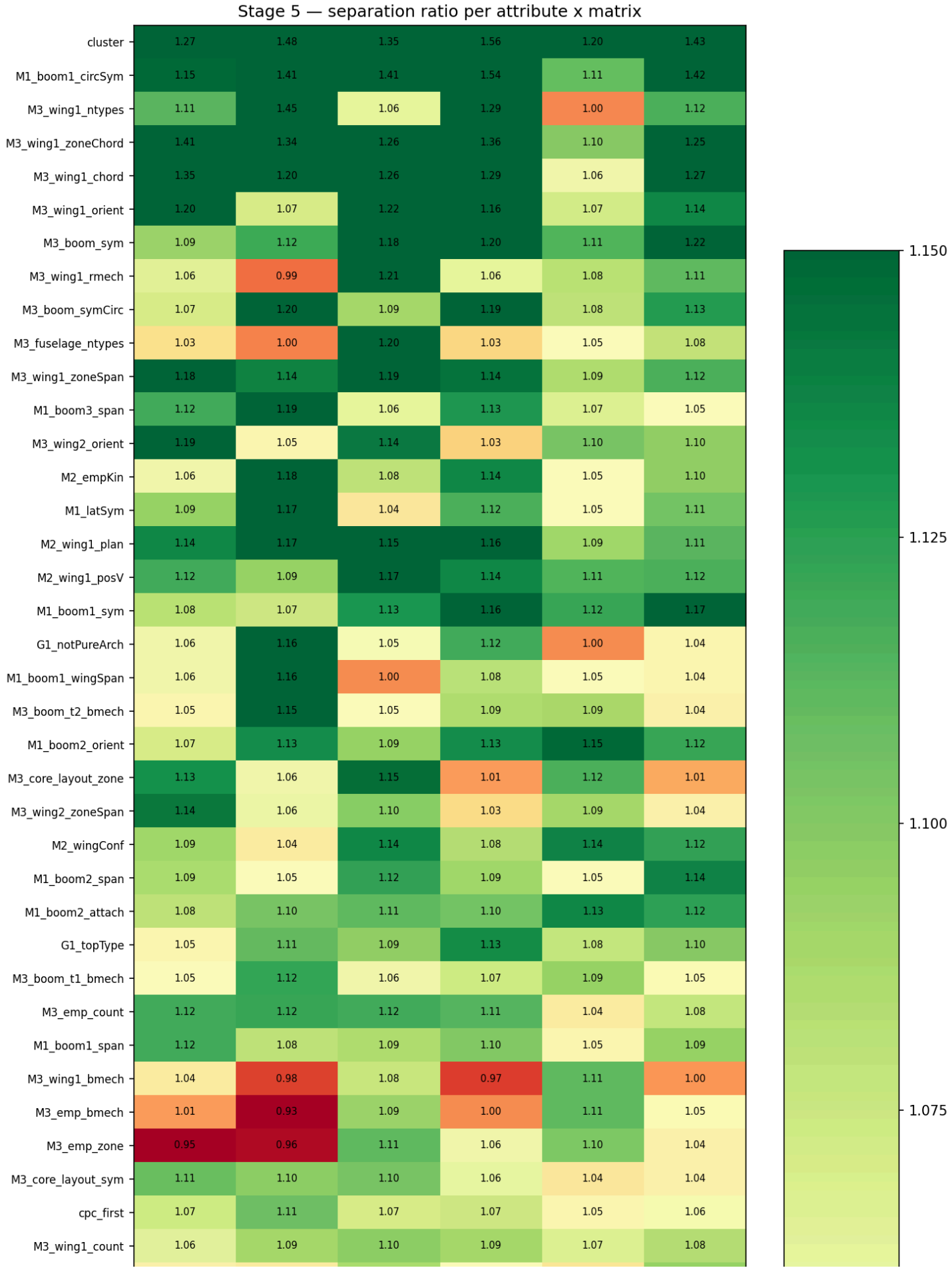
Top individual separations found (excluding the `cluster` label itself):

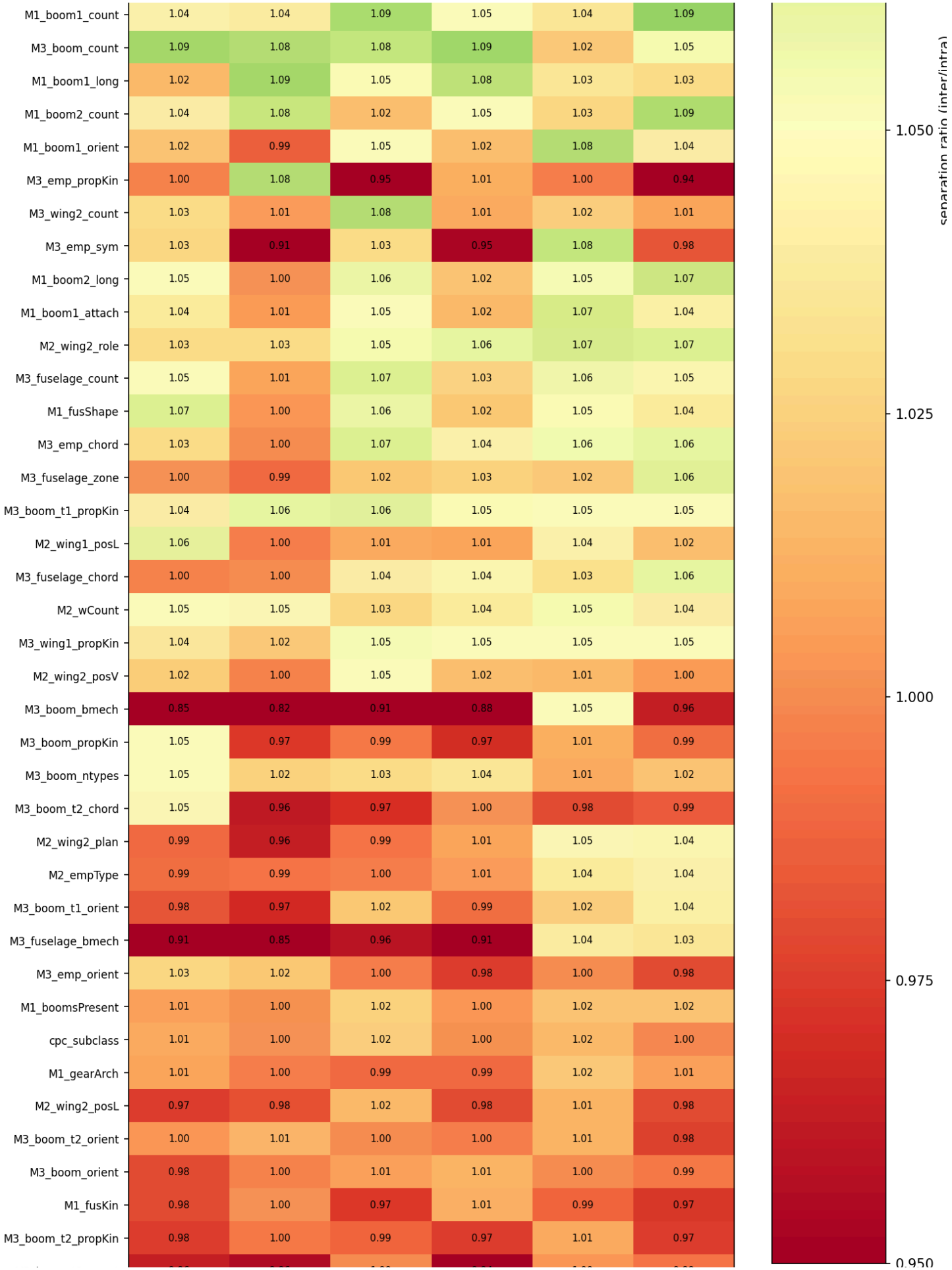
**Table 8.** Top individual attribute×layer separations (intra- vs. inter-class distance).

| Attribute | Layer | Pooling | Separation ratio | Cohen's d | p-value |
|---|---|---|---|---|---|
| **M1_boom1_circSym** | 22 | mean_patch | 1.543 | 1.248 | 0.001 |
| M1_boom1_circSym | 22 | cls | 1.412 | 1.149 | 0.001 |
| M1_boom1_circSym | 24 | mean_patch | 1.423 | 1.031 | 0.001 |
| M1_boom1_circSym | 18 | mean_patch | 1.415 | 0.911 | 0.004 |
| M3_core_layout_zone | 22 | cls | 1.146 | 0.735 | 0.005 |
| M3_wing1_chord | 22 | cls | 1.263 | 0.726 | 0.009 |

For reference, the unsupervised `cluster` label itself separates most strongly of anything tested on its own reference matrix (layer 22 mean_patch: ratio 1.560, d = 1.03, p = 0.001) — expected, since clustering is fit directly on that matrix.

Averaging Cohen’s d over all significant rows per layer×pooling, **layer 22 cls** now has the highest average effect size (mean d = 0.363), ahead of layer 22 mean_patch (0.315) and the other four matrices (0.26–0.30) — a change from the main-arch-only run, where layer 24 cls and layer 18 mean_patch led. With the full architecture set, layer 22 is the strongest performer under both pooling strategies.


Stage 5 — separation ratio per attribute x matrix
cluster 1.27 1.48 1.35 1.56 1.20 1.43
M1_boom1_circSym 1.15 1.41 1.41 1.54 1.11 1.42
M3_wing1_ntypes 1.11 1.45 1.06 1.29 1.00 1.12
M3_wing1_zoneChord 1.41 1.34 1.26 1.36 1.10 1.25
M3_wing1_chord 1.35 1.20 1.26 1.29 1.06 1.27
M3_wing1_orient 1.20 1.07 1.22 1.16 1.07 1.14
M3_boom_sym 1.09 1.12 1.18 1.20 1.11 1.22
M3_wing1_rmech 1.06 0.99 1.21 1.06 1.08 1.11
M3_boom_symCirc 1.07 1.20 1.09 1.19 1.08 1.13
M3_fuselage_ntypes 1.03 1.00 1.20 1.03 1.05 1.08
M3_wing1_zoneSpan 1.18 1.14 1.19 1.14 1.09 1.12
M1_boom3_span 1.12 1.19 1.06 1.13 1.07 1.05
M3_wing2_orient 1.19 1.05 1.14 1.03 1.10 1.10
M2_empKin 1.06 1.18 1.08 1.14 1.05 1.10
M1_latSym 1.09 1.17 1.04 1.12 1.05 1.11
M2_wing1_plan 1.14 1.17 1.15 1.16 1.09 1.11
M2_wing1_posV 1.12 1.09 1.17 1.14 1.11 1.12
M1_boom1_sym 1.08 1.07 1.13 1.16 1.12 1.17
G1_notPureArch 1.06 1.16 1.05 1.12 1.00 1.04
M1_boom1_wingSpan 1.06 1.16 1.00 1.08 1.05 1.04
M3_boom_t2_bmech 1.05 1.15 1.05 1.09 1.09 1.04
M1_boom2_orient 1.07 1.13 1.09 1.13 1.15 1.12
M3_core_layout_zone 1.13 1.06 1.15 1.01 1.12 1.01
M3_wing2_zoneSpan 1.14 1.06 1.10 1.03 1.09 1.04
M2_wingConf 1.09 1.04 1.14 1.08 1.14 1.12
M1_boom2_span 1.09 1.05 1.12 1.09 1.05 1.14
M1_boom2_attach 1.08 1.10 1.11 1.10 1.13 1.12
G1_topType 1.05 1.11 1.09 1.13 1.08 1.10
M3_boom_t1_bmech 1.05 1.12 1.06 1.07 1.09 1.05
M3_emp_count 1.12 1.12 1.12 1.11 1.04 1.08
M1_boom1_span 1.12 1.08 1.09 1.10 1.05 1.09
M3_wing1_bmech 1.04 0.98 1.08 0.97 1.11 1.00
M3_emp_bmech 1.01 0.93 1.09 1.00 1.11 1.05
M3_emp_zone 0.95 0.96 1.11 1.06 1.10 1.04
M3_core_layout_sym 1.11 1.10 1.10 1.06 1.04 1.04
cpc_first 1.07 1.11 1.07 1.07 1.05 1.06
M3_wing1_count 1.06 1.09 1.10 1.09 1.07 1.08
1.150
1.125
1.100
1.075


M1_boom1_count 1.04 1.04 1.09 1.05 1.04 1.09
M3_boom_count 1.09 1.08 1.08 1.09 1.02 1.05
M1_boom1_long 1.02 1.09 1.05 1.08 1.03 1.03
M1_boom2_count 1.04 1.08 1.02 1.05 1.03 1.09
M1_boom1_orient 1.02 0.99 1.05 1.02 1.08 1.04
M3_emp_propKin 1.00 1.08 0.95 1.01 1.00 0.94
M3_wing2_count 1.03 1.01 1.08 1.01 1.02 1.01
M3_emp_sym 1.03 0.91 1.03 0.95 1.08 0.98
M1_boom2_long 1.05 1.00 1.06 1.02 1.05 1.07
M1_boom1_attach 1.04 1.01 1.05 1.02 1.07 1.04
M2_wing2_role 1.03 1.03 1.05 1.06 1.07 1.07
M3_fuselage_count 1.05 1.01 1.07 1.03 1.06 1.05
M1_fusShape 1.07 1.00 1.06 1.02 1.05 1.04
M3_emp_chord 1.03 1.00 1.07 1.04 1.06 1.06
M3_fuselage_zone 1.00 0.99 1.02 1.03 1.02 1.06
M3_boom_t1_propKin 1.04 1.06 1.06 1.05 1.05 1.05
M2_wing1_posL 1.06 1.00 1.01 1.01 1.04 1.02
M3_fuselage_chord 1.00 1.00 1.04 1.04 1.03 1.06
M2_wCount 1.05 1.05 1.03 1.04 1.05 1.04
M3_wing1_propKin 1.04 1.02 1.05 1.05 1.05 1.05
M2_wing2_posV 1.02 1.00 1.05 1.02 1.01 1.00
M3_boom_bmech 0.85 0.82 0.91 0.88 1.05 0.96
M3_boom_propKin 1.05 0.97 0.99 0.97 1.01 0.99
M3_boom_ntypes 1.05 1.02 1.03 1.04 1.01 1.02
M3_boom_t2_chord 1.05 0.96 0.97 1.00 0.98 0.99
M2_wing2_plan 0.99 0.96 0.99 1.01 1.05 1.04
M2_empType 0.99 0.99 1.00 1.01 1.04 1.04
M3_boom_t1_orient 0.98 0.97 1.02 0.99 1.02 1.04
M3_fuselage_bmech 0.91 0.85 0.96 0.91 1.04 1.03
M3_emp_orient 1.03 1.02 1.00 0.98 1.00 0.98
M1_boomsPresent 1.01 1.00 1.02 1.00 1.02 1.02
cpc_subclass 1.01 1.00 1.02 1.00 1.02 1.00
M1_gearArch 1.01 1.00 0.99 0.99 1.02 1.01
M2_wing2_posL 0.97 0.98 1.02 0.98 1.01 0.98
M3_boom_t2_orient 1.00 1.01 1.00 1.00 1.01 0.98
M3_boom_orient 0.98 1.00 1.01 1.01 1.00 0.99
M1_fusKin 0.98 1.00 0.97 1.01 0.99 0.97
M3_boom_t2_propKin 0.98 1.00 0.99 0.97 1.01 0.97
separation ratio (inter/intra)
1.050
1.025
1.000
0.975
0.950

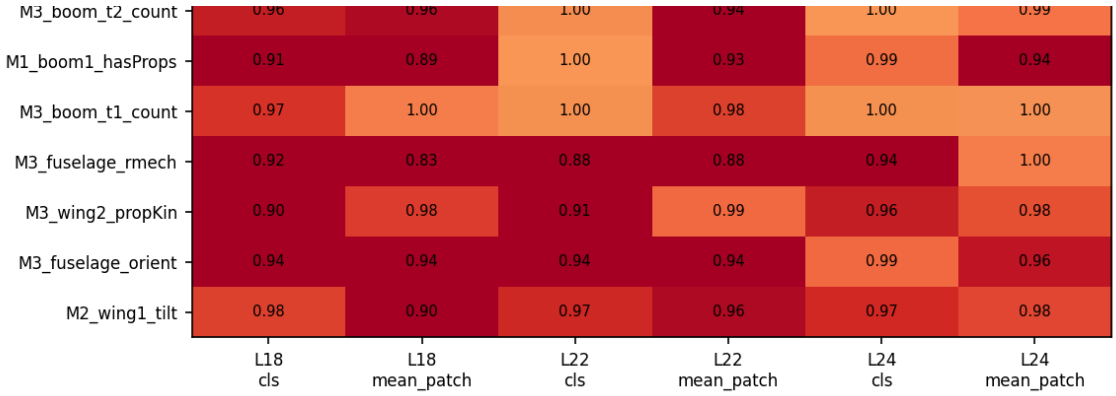



***Figure 7.*** *Separation heatmap*

**What this means for the reference-matrix choice.** This is the one test in the report that compares all 6 matrices on a taxonomy-relevant criterion, and it does not crown the configured reference matrix: layer 22 **cls** edges out layer 22 mean_patch on average attribute separation (mean d 0.363 vs. 0.315). The honest reading is that the layer-22 *depth* is validated — it wins under both pooling strategies — but the *pooling* choice within layer 22 is genuinely open. Two things temper the case for immediately rerunning Stages 3–4 on layer 22 cls: the margin is modest (≈0.05 in mean d, from a test with its own sampling noise), and this metric measures per-attribute separation, not clustering quality or confound resistance — a matrix can separate attributes slightly better while clustering less cleanly. Since only layer 22 mean_patch has been carried through the full clustering + confound analysis, the principled next iteration is to repeat Stages 3–4 on layer 22 cls (and, given the spread argument from Section 1.3, layer 24 mean_patch as a secondary candidate) and compare all three on the same criteria: silhouette, bootstrap stability, and NMI/ARI alignment versus confounds. That three-way comparison — not any single metric here — is how the reference matrix should be settled going forward. It is listed as future work rather than done inline, because each rerun regenerates the full stage-3/4/5 artefact set.

**Status:** PASSED (cluster–taxonomy alignment), with a caveat the raw verdict doesn't capture. On NMI — the metric the pipeline's gate actually checks — the best design attribute (`M3_emp_chord`, 0.260) outperforms every confound, which is what the PASS reflects. The chance-corrected view is more nuanced: `M3_emp_chord`'s signal is confirmed real (ARI 0.218 tracks its NMI closely), and the apparent `assignee` threat dissolves under correction (ARI 0.036) — but drawing perspective emerges as a genuinely strong confound (ARI 0.288, above `M3_emp_chord`'s own ARI). **Core finding:** the clusters do carry real design signal, anchored by `M3_emp_chord`, but they are not yet a *clean* design signal — perspective must be controlled for (Section 5, next steps) before cluster membership can be read as design taxonomy alone.

**CLUSTER ↔ TAXONOMY ALIGNMENT — PASSED (WITH PERSPECTIVE CAVEAT)**

---

*All tables/figures sourced from `DINOv2_eVTOL_frozen_Analysis/notebooks/11_taxonomy_structure_separation.ipynb` and its outputs in `…/Preliminary_analysis/outputs/analysis_taxonomy/`.*